

CST8917 - Final Project

Desmond Murphy - 040946131

Multi-Pattern Expense Approval Systems

Durable Functions (Code-First) vs. Logic Apps (Service Bus-First)

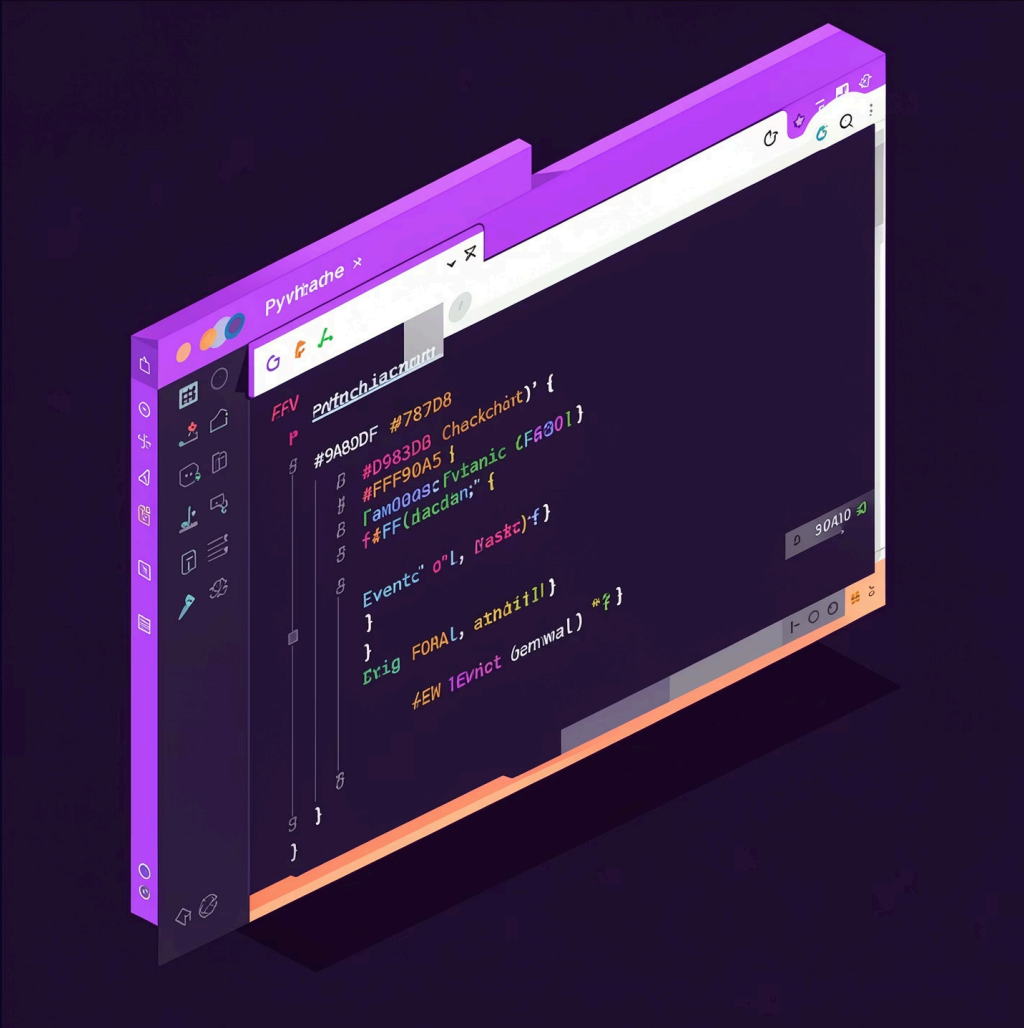
Business Rules:

- Amount < 100 = Auto-Approved
- Amount ≥ 100 = Manager Approval
 - Manager has to Approve or Deny Request
 - If Manager doesn't Approve or Deny and the Timer expires the request is Escalated
- Invalid category or Request = Validation error

How do we handle "Human-in-the-loop" states reliably at scale?



State as Code: The Durable Functions Pattern



Design Decisions

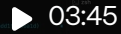
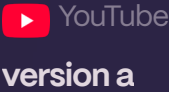
- Stateful Orchestrator manages the workflow cycle
- External HTTP trigger for manager callbacks
- Event-driven wait state using `yield`
`context.wait_for_external_event("ManagerApproval")`

Implementation Patterns

- Asynchronous HTTP Pattern for function chaining
- Human Interaction Pattern for accept/reject requests

Scenario A - Video

```
11 __main___db_init__join 11
12 __main___db_init__start 12
13 __main___db_init__join 13
14 __main___db_init__start 14
15 __main___db_init__join 15
16 __main___db_init__start 16
17 __main___db_init__join 17
18 __main___db_init__start 18
19 __main___db_init__join 19
20 __main___db_init__start 20
21 __main___db_init__join 21
22 __main___db_init__start 22
23 __main___db_init__join 23
24 __main___db_init__start 24
25 __main___db_init__join 25
26 __main___db_init__start 26
27 __main___db_init__join 27
28 __main___db_init__start 28
29 __main___db_init__join 29
30 __main___db_init__start 30
31 __main___db_init__join 31
32 __main___db_init__start 32
33 __main___db_init__join 33
34 __main___db_init__start 34
35 __main___db_init__join 35
36 __main___db_init__start 36
37 __main___db_init__join 37
38 __main___db_init__start 38
39 __main___db_init__join 39
40 __main___db_init__start 40
41 __main___db_init__join 41
42 __main___db_init__start 42
43 __main___db_init__join 43
44 __main___db_init__start 44
45 __main___db_init__join 45
46 __main___db_init__start 46
47 __main___db_init__join 47
48 __main___db_init__start 48
49 __main___db_init__join 49
50 __main___db_init__start 50
51 __main___db_init__join 51
52 __main___db_init__start 52
53 __main___db_init__join 53
54 __main___db_init__start 54
55 __main___db_init__join 55
56 __main___db_init__start 56
57 __main___db_init__join 57
58 __main___db_init__start 58
59 __main___db_init__join 59
60 __main___db_init__start 60
61 __main___db_init__join 61
62 __main___db_init__start 62
63 __main___db_init__join 63
64 __main___db_init__start 64
65 __main___db_init__join 65
66 __main___db_init__start 66
67 __main___db_init__join 67
68 __main___db_init__start 68
69 __main___db_init__join 69
70 __main___db_init__start 70
71 __main___db_init__join 71
72 __main___db_init__start 72
73 __main___db_init__join 73
74 __main___db_init__start 74
75 __main___db_init__join 75
76 __main___db_init__start 76
77 __main___db_init__join 77
78 __main___db_init__start 78
79 __main___db_init__join 79
80 __main___db_init__start 80
81 __main___db_init__join 81
82 __main___db_init__start 82
83 __main___db_init__join 83
84 __main___db_init__start 84
85 __main___db_init__join 85
86 __main___db_init__start 86
87 __main___db_init__join 87
88 __main___db_init__start 88
89 __main___db_init__join 89
90 __main___db_init__start 90
91 __main___db_init__join 91
92 __main___db_init__start 92
93 __main___db_init__join 93
94 __main___db_init__start 94
95 __main___db_init__join 95
96 __main___db_init__start 96
97 __main___db_init__join 97
98 __main___db_init__start 98
99 __main___db_init__join 99
100 __main___db_init__start 100
```



VERSION B

Decoupled Logic & Messaging: The Logic Apps Pattern



Service Bus Ingestion

Guaranteed delivery with built-in load leveling. Messages are never lost under spike traffic.



Topic-Based Routing

SQL Filters on User Properties (Status = 'Approved') route messages to the right subscribers.



Outlook Approval Connector

Native callback handling — managers approve directly from their inbox. Zero custom code.

Scenario B - Video

Code vs. Configuration: The Comparison

Dimension	Version A — Durable Functions	Version B — Logic Apps
Experience	Python / Local Dev (VS Code)	Visual Designer / Azure Portal
Testing	REST API / Azurite emulator	Service Bus Explorer / SAS tokens
Error Handling	Code-based (try / except)	Run-after conditions (infrastructure)
Observability	Application Insights (logs, traces)	Run History (visual, stakeholder-friendly)

Choose Version A if...

You have Python expertise, need complex logic, and want frictionless local testing.

Choose Version B if...

You need rapid deployment, Office 365 integration, and visual visibility for non-technical stakeholders.

Surprises & Pitfalls: Lessons Learned



SAS Token Complexity

Generating SAS tokens for REST-based testing was far more involved than standard HTTP auth. Plan extra time for this step.

01
10

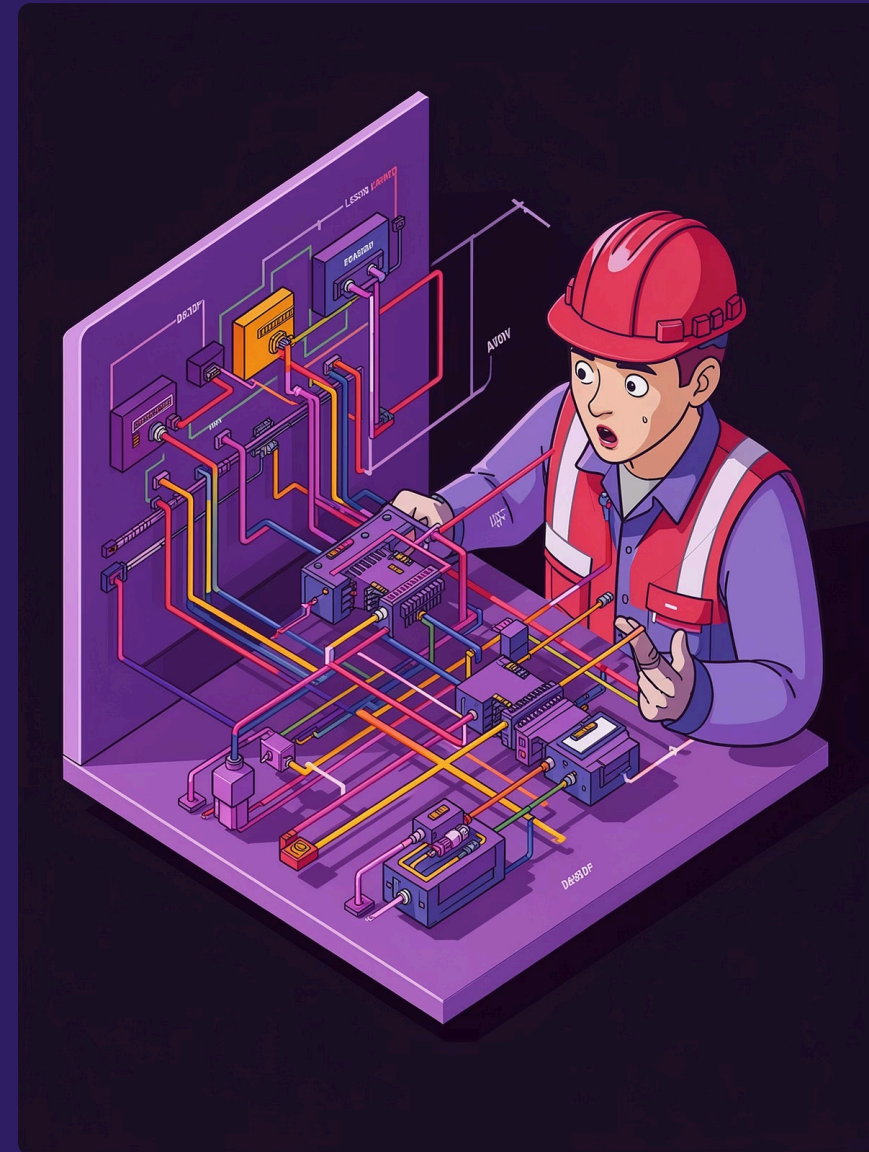
Base64 Encoding Gotchas

Service Bus payloads require careful Base64 encode/decode handling when crossing into Azure Functions. A common source of silent failures.



Do Differently: Managed Identities

Skip connection strings and SAS keys entirely. Use **Managed Identities** for all service-to-service auth — cleaner, safer, and easier to rotate.



Key Takeaways

→ **Pattern choice is a business decision**

Durable Functions suit developer-centric, code-heavy workflows. Logic Apps shine when integration speed and stakeholder visibility matter.

→ **Human-in-the-loop doesn't mean always-on**

Both patterns avoid idle compute costs — checkpointing and native connectors handle waits gracefully.

→ **Invest in identity early**

Managed Identities eliminate the SAS and connection string friction that slows down testing and deployment.